

Support Vector Machine (SVM)

Lab 1: SVM for Multi-class Classification

Objective: Implement an SVM for multi-class classification using the full iris dataset.

Materials:

- Python installed
- Jupyter Notebook or any other editor

Instructions:

1. **Setup:** Import necessary libraries.

```
import numpy as np

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import classification_report, confusion_matrix

import matplotlib.pyplot as plt

import seaborn as sns
```

2. **Data Preparation:**

```
# Load the iris dataset

iris = datasets.load_iris()

X = iris.data

y = iris.target


# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

3. **Train SVM Model:**

```
# Create and train the SVM model

model = SVC(kernel='rbf', gamma='auto')

model.fit(X_train, y_train)
```

4. Evaluate the Model:

Predict and evaluate the model

```
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

5. Visualisation:

Visualise confusion matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="YlGnBu")
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

Note: kernel='rbf' specifies that the Radial Basis Function is used as the kernel for the SVM, suitable for non-linear data. gamma='auto' sets the influence of each training example to $1/n_{\text{features}}$ (where n_{features} is the number of features in the dataset.), which might not always be optimal compared to the scale setting.

Sources

1. [science.smith.edu- Lab 15- Support Vector Machines in Python](https://science.smith.edu/Lab%2015-Support%20Vector%20Machines%20in%20Python)
2. [stackoverflow.com- How to create an SVM with multiple features for classification](https://stackoverflow.com/questions/45700181/how-to-create-an-svm-with-multiple-features-for-classification)
3. [dataquest.io- How to Implement Support Vector Machines in Python \(2023\)](https://dataquest.io/blog/how-to-implement-support-vector-machines-in-python/)
4. [geeksforgeeks.org- Image classification using Support Vector Machine \(SVM\)](https://www.geeksforgeeks.org/image-classification-using-support-vector-machine-svm/)
5. [metana.io- Support Vector Machine \(SVM\) Classifier in Python](https://metana.io/blog/support-vector-machine-svm-classifier-in-python/)
6. [youtube.com- Building Support Vector Machine Classifier from scratch](https://www.youtube.com/watch?v=7j8v8v8v8v8)

Lab 2: SVM for Image Classification

Objective: Implement an SVM for image classification using a pre-processed dataset of handwriting samples.

Materials:

- Python installed
- Jupyter Notebook or any other editor

Instructions:

1. **Setup:** Import necessary libraries.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import classification_report, confusion_matrix

import seaborn as sns
```

2. **Data Preparation:**

```
# Load the digits dataset

digits = datasets.load_digits()

X = digits.data

y = digits.target


# Split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

3. **Train SVM Model:**

```
# Create and train the SVM model

model = SVC(kernel='linear', gamma='auto')

model.fit(X_train, y_train)
```

4. Evaluate the Model:

```
# Predict and evaluate the model

y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

5. Visualise Sample Predictions:

```
# Function to plot sample predictions

def plot_sample_predictions(X_test, y_test, y_pred, n_samples=10):

    plt.figure(figsize=(10, 5))

    for index, (image, prediction, true_label) in
        enumerate(zip(X_test[:n_samples], y_pred[:n_samples],
            y_test[:n_samples])):

        plt.subplot(2, 5, index + 1)

        plt.axis('off')

        plt.imshow(image.reshape(8, 8), cmap=plt.cm.gray_r,
            interpolation='nearest')

        plt.title(f'Pred: {prediction}\nTrue: {true_label}')

    plt.show()

plot_sample_predictions(X_test, y_test, y_pred)
```

6. Visualise Confusion Matrix:

```
# Visualise confusion matrix

conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))

sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="YlGnBu")

plt.title('Confusion Matrix')

plt.xlabel('Predicted Label')

plt.ylabel('True Label')

plt.show()
```

Note: kernel='linear' configures the SVM to use a linear kernel function, ideal for linearly separable data and high-dimensional feature spaces.

It is used when the data is linearly separable, meaning that a straight line (or hyperplane in higher dimensions) can be used to separate the classes.

Sources

1. [towardsdatascience.com- Support Vector Machines explained with Python examples](https://towardsdatascience.com/support-vector-machines-explained-with-python-examples/)
2. [science.smith.edu- Lab 15- Support Vector Machines in Python](https://science.smith.edu/lab-15-support-vector-machines-in-python/)
3. [datacamp.com- Support Vector Machines with Scikit-learn Tutorial](https://datacamp.com/support-vector-machines-with-scikit-learn-tutorial/)
4. [jakevdp.github.io- In-Depth: Support Vector Machines | Python Data Science](https://jakevdp.github.io/In-Depth-Support-Vector-Machines-Python-Data-Science/)
5. [islp.readthedocs.io- Support Vector Machines](https://islp.readthedocs.io/support-vector-machines/)
6. [github.com- Practical Guide to Support Vector Machines in Python .ipynb](https://github.com/Practical-Guide-to-Support-Vector-Machines-in-Python-.ipynb)